

Integration Manual

for S32 FLS Driver

Document Number: IM11FLSASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Building the driver	6
3.1 Build Options	6
3.1.1 GCC Compiler/Assembler/Linker Options	6
3.1.2 GHS Compiler/Assembler/Linker Options	9
3.1.3 DIAB Compiler/Assembler/Linker Options	11
3.2 Files required for compilation	13
3.2.1 Fls Files	13
3.2.2 Other includes files	14
3.3 Setting up the plugins	15
3.3.1 Location of various files inside the FLS module folder	15
3.3.2 Steps to generate the configuration	16
4 Function calls to module	17
4.1 Function Calls during Start-up	17
4.2 Function Calls during Shutdown	17
4.3 Function Calls during Wake-up	17
5 Module requirements	18
5.1 Exclusive areas to be defined in BSW scheduler	18
5.1.1 Critical Region Exclusive Matrix	20
5.2 Peripheral Hardware Requirements	20
5.3 ISR to configure within AutosarOS - dependencies	21
5.4 ISR Macro	21
5.4.1 Without an Operating System	21
5.4.2 With an Operating System	21
5.5 Other AUTOSAR modules - dependencies	22
5.6 Data Cache Restrictions	22
5.7 User Mode support	22
5.7.1 User Mode configuration in the module	22
5.7.2 User Mode configuration in AutosarOS	22
5.8 Multicore support	23
5.8.1 External flash driver in multicore context	24

6 Main API Requirements	27
6.1 Main function calls within BSW scheduler	27
6.2 API Requirements	27
6.3 Calls to Notification Functions, Callbacks, Callouts	27
7 Memory allocation	28
7.1 Sections to be defined in Fls_MemMap.h	28
7.2 Linker command file	29
8 Integration Steps	30
9 External assumptions for driver	31
9.1 Application's tasks	33
9.2 External memory	33
10 Tips for FLS integration	34
10.1 Synchronous vs. Asynchronous mode	34
10.2 ECC Management on Flash	34
10.2.1 ECC Management on Qspi Flash	35



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This Integration Manual describes the integration requirements for NXP Semiconductors' AUTOSAR Flash Driver for S32.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DET	Default Error Tracer
ECC	Error Correcting Code
VLE	Variable Length Encoding
N/A	Not Available
MCU	Microcontroller Unit
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
FEE	Flash EEPROM Emulation
FLS	Flash
RTD	Real Time Drivers
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of FLS Driver	AUTOSAR Release 4.4.0
2	Reference Manual	S32G2 Reference Manual, Rev. 7, February 2023
		S32G3 Reference Manual, Rev. 3, February 2023
		S32R45 Reference Manual, Rev. 3, 12/2021
3	Errata	S32G2: Mask Set Errata for Mask 0P77B v3.0
		S32G2: Mask Set Errata for Mask 1P77B v1.0
		S32G3: Mask Set Errata for Mask 1P72B v2.1
		S32R45: Mask Set Errata for Mask P57D, Rev. 2.0
4	Data Sheet	S32G2 Data Sheet, Rev 6, December 2022
		S32G3 Data Sheet, Rev 2, RC, February 2023
		VR5510 Data Sheet, Rev 5, April 2022
		S32R45 Data Sheet, Rev. 3, RC — 11/2022

Chapter 3

Building the driver

- [Build Options](#)
- [Files required for compilation](#)
- [Setting up the plugins](#)

This section describes the source files and various compilers, linker options used for building the driver. It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

- [GCC Compiler/Assembler/Linker Options](#)
- [GHS Compiler/Assembler/Linker Options](#)
- [DIAB Compiler/Assembler/Linker Options](#)

The RTD driver files are compiled using:

- NXP GCC 9.2.0 20190812 (Build 1649 Revision gaf57174)
- Green Hills Multi 7.1.6d / Compiler 2020.1.4
- Wind River Diab Compiler 7.0.3

The compiler, assembler, and linker flags used for building the driver are explained below.

The TS_T40D11M40I2R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D11 identifies S32 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

3.1.1 GCC Compiler/Assembler/Linker Options

3.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpv=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioned -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned
-fomit-frame-pointer	Omit the frame pointer in functions that don't need one. This avoids the instructions to save, set up and restore the frame pointer; on many targets it also makes an extra register available.

Compiler Option	Description
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	This option is only used to build test for generation Ram/↔ Stack size report. Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	This option is only used to build test for generation Ram/↔ Stack size report. Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DGCC	Predefine GCC as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.↔ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT↔ RT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.1.2 GCC Assembler Options

Assembler Option	Description
-X assembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

3.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-entry=Reset_Handler	Specifies that the program entry point is Reset_Handler
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library

3.1.2 GHS Compiler/Assembler/Linker Options

3.1.2.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-thumb	Selects generating code that executes in Thumb state
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-C99	Use (strict ISO) C99 standard (without extensions)
-ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)
-Osize	Optimize for size
-gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory

Compiler Option	Description
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
-unsigned_chars	Let the type char be unsigned, like unsigned char
-unsigned_fields	Bitfields declared with an integer type are unsigned
-no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
-no_exceptions	Disables C++ support for exception handling
-no_slash_comment	C++ style // comments are not accepted and generate errors
-prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
-incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DGHS	Predefine GHS as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.↵c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT↵RT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.2.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed

Assembler Option	Description
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

3.1.2.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWARF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for the modules in this library is provided in the src/libstartup directory
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-nostartfiles	Controls the start files to be linked into the executable

3.1.3 DIAB Compiler/Assembler/Linker Options

3.1.3.1 DIAB Compiler Options

Compiler Option	Description
-tARMCORTEXM7MG:simple	Selects target processor (hardware single-precision, software double-precision floating-point)
-mthumb	Selects generating code that executes in Thumb state
-std=c99	Follows the C99 standard for C
-Oz	Like -O2 with further optimizations to reduce code size
-g	Generates DWARF 4.0 debug information
-fstandalone-debug	Emits full debug info for all types used by the program

Compiler Option	Description
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wsign-compare	Produce warnings when comparing signed type with unsigned type
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-Wunknown-pragmas	Issues a warning for unknown pragmas
-Wundef	Warns if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wextra	Enables some extra warning flags that are not enabled by '-Wall'
-Wall	Enables all of the most useful warnings (for historical reasons this option does not literally enable all warnings)
-pedantic	Emits a warning whenever the standard specified by the -std option requires a diagnostic
-Werror=implicit-function-declaration	Generates an error whenever a function is used before being declared
-fno-common	Compile common globals like normal definitions
-fno-signed-char	Char is unsigned
-fno-trigraphs	Do not process trigraph sequences
-V	Displays the current version number of the tool suite
-c	Stop after assembly and produce an object file for each source file
-DS32XX	Predefine S32XX as a macro, with definition 1
-DS32G2XX	Predefine S32G2XX as a macro, with definition 1
-DS32G3XX	Predefine S32G3XX as a macro, with definition 1
-DS32R45	Predefine S32R45 as a macro, with definition 1
-DDIAB	Predefine DIAB as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.3.2 DIAB Assembler Options

Assembler Option	Description
-mthumb	Selects generating code that executes in Thumb state
-Xpreprocess-assembly	Invokes C preprocessor on assembly files before running the assembler
-Xassembly-listing	Produces an .lst assembly listing file
-c	Stop after assembly and produce an object file for each source file

3.1.3.3 DIAB Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
linker_script_file.dld	Use linker_script_file.dld as the linker script. This script replaces the default linker script (rather than adding to it)
-m30	m2 + m4 + m8 + m16
-Xstack-usage	Gathers and display stack usage at link time
-Xpreprocess-lecl	Perform pre-processing on linker scripts
-Llibrary_path	Points to the libraries location for ARMV7EMMG to be used for linking
-lc	Links with the standard C library
-lm	Links with the math library

3.2 Files required for compilation

- This section describes the include files required to compile, assemble and link the AUTOSAR Flash Driver for S32 microcontrollers.
- To avoid integration of incompatible files, all the include files from other modules shall have the same AR_↔ MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

3.2.1 Fls Files

- Include file:
 - Fls_TS_T40D11M40I2R0\include\Fls.h
 - Fls_TS_T40D11M40I2R0\include\Fls_Api.h
 - Fls_TS_T40D11M40I2R0\include\Fls_IPW.h
 - Fls_TS_T40D11M40I2R0\include\Fls_Types.h
 - Fls_TS_T40D11M40I2R0\include\Qspi_IP.h
 - Fls_TS_T40D11M40I2R0\include\Qspi_IP_Types.h
 - Fls_TS_T40D11M40I2R0\include\Qspi_Ip_Common.h
 - Fls_TS_T40D11M40I2R0\include\Qspi_Ip_Controller.h
 - Fls_TS_T40D11M40I2R0\include\Qspi_Ip_HwAccess.h
 - Fls_TS_T40D11M40I2R0\include\Qspi_Ip_Hyperflash.h

- Fls_TS_T40D11M40I2R0\include\Qspi_Ip_HyperflashRegs.h
- Fls_TS_T40D11M40I2R0\include\Qspi_Ip_HyperflashTypes.h
- Source files:
 - Fls_TS_T40D11M40I2R0\src\Fls.c
 - Fls_TS_T40D11M40I2R0\src\Fls_IPW.c
 - Fls_TS_T40D11M40I2R0\src\Qspi_Ip.c
 - Fls_TS_T40D11M40I2R0\src\Qspi_Ip_Controller.c
 - Fls_TS_T40D11M40I2R0\src\Qspi_Ip_Sfdp.c
 - Fls_TS_T40D11M40I2R0\src\Qspi_Ip_Hyperflash.c

Note

These files should be generated by the user using a configuration/generation tool

- Fls_Cfg.h
- Fls_Cfg.c
- Fls_PBcfg.h
- Fls_PBcfg.c
- Fls_Cfg_Defines.h
- Qspi_Ip_Features.h
- Qspi_Ip_CfgDefines.h
- Qspi_Ip_Cfg.h
- Qspi_Ip_PBcfg.h
- Qspi_Ip_PBcfg.c

3.2.2 Other includes files

3.2.2.1 Files from MemIf folder

- MemIf_TS_T40D11M40I2R0\include\MemIf_Types.h

3.2.2.2 Files from BaseNXP folder

- BaseNXP_TS_T40D11M40I2R0\include\Compiler.h
- BaseNXP_TS_T40D11M40I2R0\include\Compiler_Cfg.h
- BaseNXP_TS_T40D11M40I2R0\include\ComStack_Types.h
- BaseNXP_TS_T40D11M40I2R0\include\Fls_MemMap.h
- BaseNXP_TS_T40D11M40I2R0\include\Mcal.h
- BaseNXP_TS_T40D11M40I2R0\include\Platform_Types.h
- BaseNXP_TS_T40D11M40I2R0\include\Std_Types.h
- BaseNXP_TS_T40D11M40I2R0\include\Reg_eSys.h
- BaseNXP_TS_T40D11M40I2R0\include\Soc_Ips.h
- BaseNXP_TS_T40D11M40I2R0\include\Reg_Macros.h
- BaseNXP_TS_T40D11M40I2R0\include\SilRegMacros.h
- BaseNXP_TS_T40D11M40I2R0\header\S32G274A.h

3.2.2.3 Files from Det folder

- Det_TS_T40D11M40I2R0\include\Det.h

3.2.2.4 Files from Platform folder:

- Platform_TS_T40D11M40I2R0\include\Platform.h

3.2.2.5 Files from Rte folder

- Rte_TS_T40D11M40I2R0\include\SchM_Fls.h

3.2.2.6 Files from Rm folder

- Rm_TS_T40D11M40I2R0\include\CDD_Rm.h

3.3 Setting up the plugins

The Flash Driver was designed to be configured by using the EB Tresos Studio (version 27.1.0 b200625-0900 or later)

3.3.1 Location of various files inside the FLS module folder

- VSMD (Vendor Specific Module Definition) file in EB Tresos Studio XDM format:
 - Fls_TS_T40D11M40I2R0\config\Fls.xdm
- VSMD (Vendor Specific Module Definition) file(s) in AUTOSAR compliant EPD format:
 - Fls_TS_T40D11M40I2R0\autosar\Fls_<subderivative_name>.epd
- Code Generation Templates for variant aware parameters:
 - Fls_TS_T40D11M40I2R0\generate_PB\include\Fls_PBcfg.h
 - Fls_TS_T40D11M40I2R0\generate_PB\src\Fls_PBcfg.c
 - Fls_TS_T40D11M40I2R0\generate_PB\src\Qspi_Ip_PBcfg.c
 - Fls_TS_T40D11M40I2R0\generate_PB\include\Qspi_Ip_PBcfg.h
- Code Generation Templates for parameters without variation points:
 - Fls_TS_T40D11M40I2R0\generate_PC\include\Fls_Cfg.h
 - Fls_TS_T40D11M40I2R0\generate_PC\include\Qspi_Ip_Cfg.h
 - Fls_TS_T40D11M40I2R0\generate_PC\include\Qspi_Ip_Features.h
 - Fls_TS_T40D11M40I2R0\generate_PC\include\Qspi_Ip_CfgDefines.h
 - Fls_TS_T40D11M40I2R0\generate_PC\src\Fls_Cfg.c

3.3.2 Steps to generate the configuration

1. Copy the module folders:
 - BaseNXP_TS_T40D11M40I2R0
 - Det_TS_T40D11M40I2R0
 - EcuM_TS_T40D11M40I2R0
 - Fls_TS_T40D11M40I2R0
 - Rm_TS_T40D11M40I2R0
 - MemIf_TS_T40D11M40I2R0
 - Os_TS_T40D11M40I2R0
 - Platform_TS_T40D11M40I2R0
 - Resource_TS_T40D11M40I2R0
 - Rte_TS_T40D11M40I2R0into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files.

Chapter 4

Function calls to module

- [Function Calls during Start-up](#)
- [Function Calls during Shutdown](#)
- [Function Calls during Wake-up](#)

4.1 Function Calls during Start-up

Fls shall be initialized during STARTUP phase of EcuM initialization.

The API to be called for this is Fls_Init().

4.2 Function Calls during Shutdown

None.

4.3 Function Calls during Wake-up

None.

Chapter 5

Module requirements

- Exclusive areas to be defined in BSW scheduler
- unavailable_exclusive_areas
- Peripheral Hardware Requirements
- ISR to configure within AutosarOS - dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restrictions
- User Mode support
- Multicore support

5.1 Exclusive areas to be defined in BSW scheduler

In the current implementation, Fls is using the services of Schedule Manager (SchM) for entering and exiting the exclusive areas. The following critical regions are used in the Fls driver:

FLS_EXCLUSIVE_AREA_10 is used in function Fls_Erase to protect the updates for:

- Fls_eJobResult
- Fls_u32JobSectorIt
- Fls_u32JobSectorEnd
- Fls_eJob
- Fls_u8JobStart
- Fls_eJobResult

FLS_EXCLUSIVE_AREA_11 is used in function Fls_Write to protect the updates for:

- Fls_eJobResult
- Fls_u32JobSectorIt
- Fls_u32JobSectorEnd
- Fls_u32JobAddrEnd
- Fls_u32JobAddrIt
- Fls_pJobDataSrcPtr
- Fls_eJob
- Fls_u8JobStart
- Fls_eJobResult

FLS_EXCLUSIVE_AREA_12 is used in function Fls_Read to protect the updates for:

- Fls_eJobResult
- Fls_u32JobSectorIt
- Fls_u32JobSectorEnd
- Fls_u32JobAddrIt
- Fls_u32JobAddrEnd
- Fls_pJobDataDestPtr
- Fls_u8JobStart
- Fls_eJobResult

FLS_EXCLUSIVE_AREA_13 is used in function Fls_Compare to protect the updates for:

- Fls_eJobResult
- Fls_u32JobSectorIt
- Fls_u32JobSectorEnd
- Fls_u32JobAddrIt
- Fls_u32JobAddrEnd
- Fls_pJobDataSrcPtr
- Fls_eJob
- Fls_u8JobStart
- Fls_eJobResult

FLS_EXCLUSIVE_AREA_14 is used in function Fls_BlankCheck to protect the updates for:

- Fls_eJobResult

Module requirements

- Fls_u32JobSectorIt
- Fls_u32JobAddrIt
- Fls_u32JobAddrEnd
- Fls_eJob
- Fls_u8JobStart
- Fls_eJobResult

The critical regions from interrupts are grouped in “Interrupt Service Routines Critical Regions (composed diagram)”. If an exclusive area is “exclusive” with the composed “Interrupt Service Routines Critical Regions (composed diagram)” group, it means that it is exclusive with each one of the ISR critical regions.

5.1.1 Critical Region Exclusive Matrix

- Below is the table depicting the exclusivity between different critical region IDs from the FLS driver.
- If there is an “X” in a table, it means that those 2 critical regions cannot interrupt each other.

FLS_EXCLUSIVE_AREA	AREA_10	AREA_11	AREA_12	AREA_13	AREA_14
AREA_10		X	X	X	X
AREA_11	X		X	X	X
AREA_12	X	X		X	X
AREA_13	X	X	X		X
AREA_14	X	X	X	X	

5.2 Peripheral Hardware Requirements

The FLS driver uses the external flash memory peripheral (QuadSPI). For more details about peripherals and their structure refer to the S32CC user manual.

Based on the configured type of sectors (external or internal), the FLS driver will spread a job across both internal and external memories. The hardware IP used to complete a job section is decided based on the current sector type. For consistent operations it is recommended to configure and allocate jobs on a single type of memory and avoid mixing internal and external sectors.

If external sectors are to be used, the FLS driver will configure the QuadSPI IP. Prior to FLS driver initialization, the QuadSPI IP should be enabled (clock, power) as required by the application, to a state in which the the FLS driver can configure and use it. All platform settings or specific external memory settings are out of FLS driver scope and are expected to be completed before any job is attempted.

Given the large diversity of implementation solutions in the external memories, the following operations are not fully implemented at the FLS driver level for external sectors:

- External memory calibration/data learning

- External memory error check.

In order to aid implementation, callout functions and external APIs are provided in which the application could choose the best approach adapted to attached external memory. More details are provided in the following sections and User Manual.

5.3 ISR to configure within AutosarOS - dependencies

None.

5.4 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

5.4.1 Without an Operating System The macro *USING_OS_AUTOSAROS* must not be defined.

5.4.1.1 Using Software Vector Mode

The macro *USE_SW_VECTOR_MODE* must be defined and the ISR macro is defined as:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

5.4.1.2 Using Hardware Vector Mode

The macro *USE_SW_VECTOR_MODE* must not be defined and the ISR macro is defined as:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, the drivers' interrupt handlers must also handle the context save and restore.

5.4.2 With an Operating System Please refer to your OS documentation for description of the ISR macro.

5.5 Other AUTOSAR modules - dependencies

- **Base:** The BASE module contains the common files/definitions needed by all RTD modules.
- **Det:** The DET module is used for enabling Development error detection. The API function used are Det_ReportError() or Det_ReportRuntimeError() or Det_ReportTransientFault(). The activation / deactivation of Development error detection is configurable using the "FlsDevErrorDetect" or "FlsRuntimeErrorDetect" configuration parameter.
- **Rte:** The RTE module is needed for implementing data consistency of exclusive areas that are used by FLS module.
- **MemIf:** This module allows the NVRAM manager to access several memory abstraction modules.
- **Resource:** Resource module is used to select microcontroller's derivatives.
- **EcuC:** The ECUC module is used for ECU configuration. RTD modules need ECUC to retrieve the variant information.
- **Mcl:** This module provides service for Cache operation.
- **Os:** The OS module is used for OS configuration. RTD modules need OS to retrieve the application information.

5.6 Data Cache Restrictions

- QuadSPI's AHB region (0-1FFF_FFFFh) is noncacheable

5.7 User Mode support

- [User Mode configuration in the module](#)
- [User Mode configuration in AutosarOS](#)

5.7.1 User Mode configuration in the module

No special measures need to be taken to run Fls module from user mode. The Fls driver code can be executed at any time from both supervisor and user mode.

5.7.2 User Mode configuration in AutosarOS

When User mode is enabled, the driver may has the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header <IpName>_IpTrustedFunctions.h. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter [User Mode configuration in the module](#) for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

```
Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)
```

That is the result of macro expansion `OsIf_Trusted_Call` in driver code:

```
#define OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)
```

So, the following steps need to be done in AutosarOS:

- Ensure `MCAL_ENABLE_USER_MODE_SUPPORT` macro is defined in the build system or somewhere global.
- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to be visible in `Os.h` for the driver to call them. They will do the marshalling of the parameters and call `CallTrustedFunction()` in OS specific manner.
- `CallTrustedFunction()` will switch to privileged mode and call `TRUSTED_<Function_Name>()`.
- `TRUSTED_<Function_Name>()` function is also defined and declared in Integration/User code. It will un-marshalling of the parameters to call `<Function_Name>()` of driver. The `<Function_Name>()` functions are already defined in driver and declared in `<IpName>_Ip_TrustedFunctions.h`. This header should be included in OS for OS call and indexing these functions.

See the sequence chart below for an example calling `Linflexd_Uart_Ip_Init_Privileged()` as a trusted function.

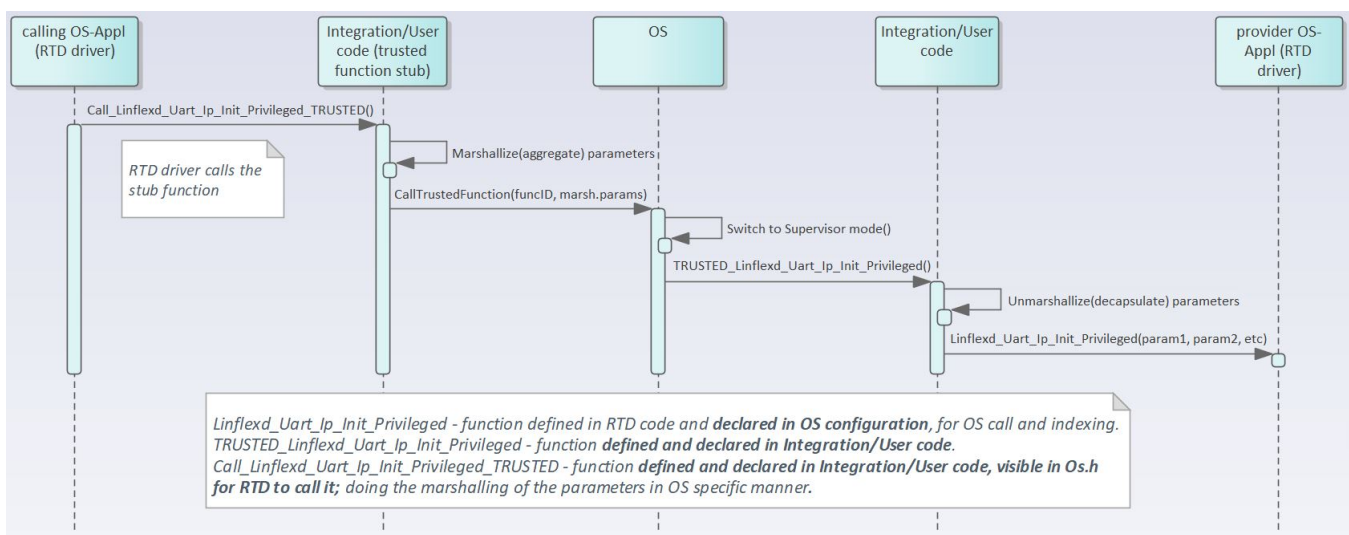


Figure 5.1 Example sequence chart for calling `Linflexd_Uart_Ip_Init_Privileged` as trusted function

5.8 Multicore support

- The FLS implements "Autosar 4.4 MCAL Multi-Core Distribution" according to Type I. For more details, please refer to AUTOSAR_EXP_BSWDistributionGuide.
- The FLS driver supports multicore synchronization if "FlsMCoreEnable" is enabled. Vendor-specific Sema4 multi-core support
- When the synchronization is enabled, the Multicore related parameters (semaphore addresses, timeout values, core number) have to be configured accordingly on both cores
- The synchronization feature arbitrates the access to the flash resource, using the configured hardware semaphores to ensure there is no access conflict from two different cores on the same hardware registers
 - The core which wins the arbitration will access the flash resource and start the flash job in hardware
 - The core which loses the arbitration will mark it's current job as pending until the controller is idle or until timeout occurs

Module requirements

Warning

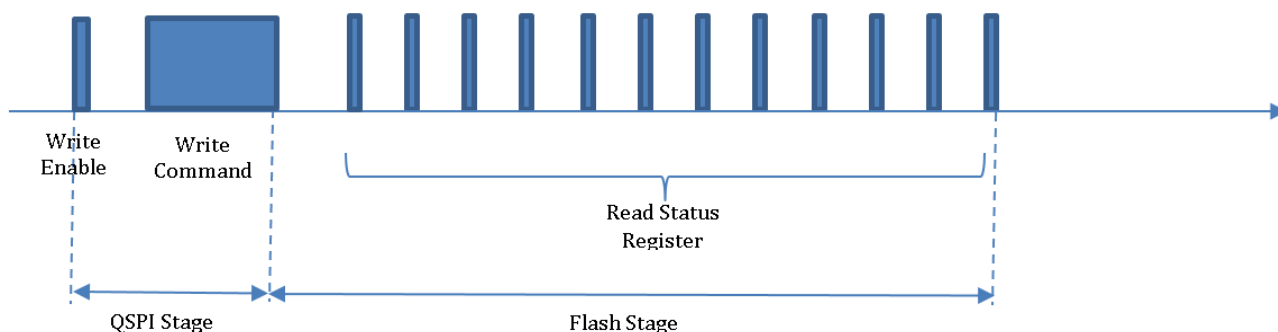
- If more than one core perform the initialization, users are responsible for ensuring configuration synchronization between cores.
- The configuration of one core should be compatible with the other cores.
- For instance, if two cores work with an external device memories through a QuadSPI hardware instance, it is impossible when a core work in single data rate mode and the other core works in double data rate mode.

- [External flash driver in multicore context](#)

5.8.1 External flash driver in multicore context

5.8.1.1 Background

- External flash operations consist of two stages. In stage 1 (QSPI stage) the command is sent to the external flash via QSPI controller. In stage 2 (Flash Stage) the actual operation takes place in the external flash and the MCU must poll the status of the operation to find when it ends. Flash stage only applies to Write and Erase operations.



- In multicore context the general restriction is that no two operations (read, write, erase) may be performed in parallel. The QSPI stages of different operations may not overlap because the QSPI module can only do one transmission at a time. For write and erase operations this restriction usually includes the flash stage too, because most serial flashes do not offer read-while-write features. So both stages depicted above must be performed as one atomic operation (not interruptible by other cores).

5.8.1.2 FLS Driver Multicore approach

- QSPI / external flash can be protected against concurrent accesses by an XRDC semaphore [hereinafter only named semaphore]. Each core that wants to make any flash operations (read, write or erase) through QSPI must first take the semaphore. This is done internally by the FLS driver. The semaphore is released at the end of the operation
- To further guard against interference, XRDC must be set to allow write access to QSPI registers only if the semaphore is owned by the respective domain. This ensures the current flash operation cannot be impacted by writes to QSPI registers performed by another (faulty) core
- For write and erase operations the semaphore must be held for the entire duration of the operation, including the flash stage. Starting another operation during the flash stage would result in the command being ignored by the flash device (and dummy data provided in case of read). Read operations also need to own the semaphore, to avoid QSPI contentions or RWW restriction violations in the flash device
- For large operations, which are internally split by the driver in several smaller operations, the semaphore is held for the duration of the entire operation.
- The semaphore is taken on first come – first served basis, there is no arbitration between cores. This can potentially lead to denial of service for an indeterminate amount of time for a core, if other cores continuously perform flash operations and grab the semaphore first

5.8.1.3 AHB Reads

- It is not recommended to perform AHB reads directly from the application, as they may interfere with normal FLS driver functionality. QSPI can only perform one operation at a time, so attempting AHB reads while the module is busy may cause the current operation to fail. It is recommended to do all reads using the FLS driver's `Fls_Read()` function. This must also be taken into account while debuggers are attached which may also read from flash
- If the application must perform AHB reads, the same XRDC semaphore used by the FLS driver must be used to guard the AHB reads. This can not be enforced by XRDC, as the semaphore only controls write permissions. So in this case it is the software's responsibility to ensure the semaphore is owned before AHB reads, failing to do so could potentially cause flash operations to fail
- If a core only needs to perform read operations during the runtime stage, it can use AHB reads and benefit from the security features of the XRDC, which can ensure the core only has access to the allocated memory range and also can forbid the core's access to the QSPI registers, thus ensuring no interference with other cores' operations. The requirement to use the semaphore still applies

5.8.1.4 Read while write

- Read While Write is not supported, as AUTOSAR only allows one flash operation at a time. This cannot be changed without changing the AUTOSAR API, since the function `Fls_GetJobResult()` and the job end notifications have no parameters so it is not possible to distinguish between multiple parallel jobs

5.8.1.5 Error recovery

- If one of the cores crashes while owning the semaphores it is possible to recover the semaphore by forcefully resetting it from outside the Fls driver. This mechanism allows either individual or all gates to be reset by a separate domain. If the application has no other way of knowing that another core has crashed, a timeout can be used. The timeout value depends on the application (one core may experience delays caused by multiple operations started from the other cores, as there is no arbitration when taking the semaphore)
- After the semaphore reset any core can initiate flash operations
- If the flash device was left in a busy state after semaphore reset, the driver will automatically issue a software reset command before the next flash operation

Chapter 6

Main API Requirements

- [Main function calls within BSW scheduler](#)
- [API Requirements](#)
- [Calls to Notification Functions, Callbacks, Callouts](#)

6.1 Main function calls within BSW scheduler

Fls_MainFunction (call rate depends on target application, i.e. how fast the data needs to be read/written/compared into Flash memory).

6.2 API Requirements

None.

6.3 Calls to Notification Functions, Callbacks, Callouts

The FLS driver provides notifications that are user configurable:

Notification	Usage
FlsJobEndNotification	Usually routed to Fee module
FlsJobErrorNotification	Usually routed to Fee module
FlsMCoreTimeoutNotification	Notify the multi core timeout in case FlsMCoreEnable node is enabled
FlsQspiInitCallout	Perform additional checks and configurations on the external memory at the end of FLS initialization
FlsQspiResetCallout	Perform reset or cancel on external memory, depending on the available reset mechanisms
FlsQspiErrorCheckCallout	Check any errors occurred in external memories during erase or program operations
FlsQspiEccCheckCallout	Interrogate the ECC status of the memory after each read operation

Chapter 7

Memory allocation

- [Sections to be defined in Fls_MemMap.h](#)
- [Linker command file](#)

7.1 Sections to be defined in Fls_MemMap.h

Index	Section name	Type of section	Description
1	FLS_START_SEC_CODE	Code	Start of memory Section for Code.
	FLS_STOP_SEC_CODE		End of above section.
2	FLS_START_SEC_CONFIG_DATA_8	Configuration Data	Start of memory Section for Configuration Data that have to be aligned to 8 bit.
	FLS_STOP_SEC_CONFIG_DATA_8		End of above section.
3	FLS_START_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start of Memory Section for Config Data. Used for variables, constants, structure, array and unions when SIZE (alignment) does not fit the criteria of 8,16 or 32 bit. For instance used for variables of unknown size
	FLS_STOP_SEC_CONFIG_DATA_UNSPECIFIED		End of above section.
4	FLS_START_SEC_CONST_32	Constant	Data Used for constants that have to be aligned to 32 bit.
	FLS_STOP_SEC_CONST_32		End of above section.
5	FLS_START_SEC_CONST_UNSPECIFIED	Constant Data	The parameters that are not variant aware shall be stored in memory section for constants.
	FLS_STOP_SEC_CONST_UNSPECIFIED		End of above section.
6	FLS_START_SEC_VAR_INIT_8	Variables	Start of memory Section for Variable declaration that have to be aligned to 8 bit.
	FLS_STOP_SEC_VAR_INIT_8		End of above section.

Index	Section name	Type of section	Description
7	FLS_START_SEC_VAR_INIT_↔ UNSPECIFIED	Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit. These variables are never cleared and never initialized by start-up code
	FLS_STOP_SEC_VAR_INIT_↔ UNSPECIFIED		End of above section.
8	FLS_START_SEC_VAR_CLEA↔ RED_8	Variables	Start of Memory Section for Variable 8 bits. These variables are cleared to zero by start-up code.
	FLS_STOP_SEC_VAR_CLEAR↔ ED_8		End of above section.
9	FLS_START_SEC_VAR_CLEA↔ RED_16	Variables	Start of Memory Section for Variable 16 bits. These variables are cleared to zero by start-up code.
	FLS_STOP_SEC_VAR_CLEAR↔ ED_16		End of above section.
10	FLS_START_SEC_VAR_CLEA↔ RED_32	Variables	Start of Memory Section for Variable 32 bits. These variables are cleared to zero by start-up code.
	FLS_STOP_SEC_VAR_CLEAR↔ ED_32		End of above section.
11	FLS_START_SEC_VAR_CLEA↔ RED_BOOLEAN	Variables	Start of memory Section for Variables with type boolean. These variables are cleared to zero by start-up code.
	FLS_STOP_SEC_VAR_CLEAR↔ ED_BOOLEAN		End of above section.
12	FLS_START_SEC_VAR_CLEA↔ RED_UNSPECIFIED	Variables	Start of memory Section for Variables. Used for variables, constants, structure, array and unions when SIZE (alignment) does not fit the criteria of 8, 16 or 32 bit. For instance used for variables of unknown size. These variables are cleared to zero by start-up code.
	FLS_STOP_SEC_VAR_CLEAR↔ ED_UNSPECIFIED		End of above section.

7.2 Linker command file

Memory shall be allocated for every section defined in the driver's "<Module>"_MemMap.h.



Chapter 8

Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section [Files Required for Compilation](#)
2. Allocate the proper memory sections in the driver's memory map header file ("`<Module>_MemMap.h`") and linker command file. For more details refer to section [Sections to be defined in `<Module>\_MemMap.h`](#)
3. Compile & build the module with all the dependent modules. For more details refer to section [Building the Driver](#)

Chapter 9

External assumptions for driver

The section presents requirements that must be complied with when integrating the FLS driver into the application.

External Assumption Req ID	External Assumption Text
SWS_Fls_00214	The FLS module shall only load the access code to the RAM if the access code cannot be executed out of flash ROM. Note: There is a global configuration parameter to select if access codes are loaded into RAM or not. This requirement only apply for internal flash.
SWS_Fls_00240	The FLS module's environment shall only call the function Fls_Read after the FLS module has been initialized. Note: Out of scope sMcal
SWS_Fls_00038	When a job has been initiated, the FLS module's environment shall call the function Fls_MainFunction cyclically until the job is finished. Note: Out of scope sMcal
SWS_Fls_00260	API function - Header File - Description - Det_ReportRuntimeError - Det.h - Service to report runtime errors. If a callout has been configured then this callout shall be called. - Note: Out of scope sMcal
SWS_Fls_00261	API function - Header File - Description - Det_ReportError - Det.h - Service to report development errors. - Note: Out of scope sMcal
SWS_Fls_00262	Service name: - Fee_JobEndNotification - Syntax: - void Fee_JobEndNotification(void) - Sync/Async: - Synchronous - Reentrancy: - Don't care - Parameters (in): - None - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - This callback function is called when a job has been completed with a positive result. - Available via: - Fee.h - Note: Configurable interface (Fee_JobEndNotification() callback).
SWS_Fls_00263	Service name: - Fee_JobErrorNotification - Syntax: - void Fee_JobErrorNotification(void) - Sync/Async: - Synchronous - Reentrancy: - Don't care - Parameters (in): - None - Parameters (inout): - None - Parameters (out): - None - Return value: - None - Description: - This callback function is called when a job has been canceled or finished with negative result. - Available via: - Fee.h - Note: Configurable interface (Fee_JobErrorNotification() callback).
EA_RTD_00071	If interrupts are locked, a centralized function pair to lock and unlock interrupts shall be used.
EA_RTD_00072	The option "Fls Hardware Timeout Handling" shall be enabled in the FLS driver configuration and the timeout value shall be configured to fit the application timing.

External Assumption Req ID	External Assumption Text
EA_RTD_00080	The integrator shall assure the execution of code from system RAM when flash memory configurations need to be change (i.e. PFCR control fields of PFLASH memory need to be change) .
EA_RTD_00082	When caches are enabled and data buffers are allocated in cacheable memory regions the buffers involved in DMA transfer shall be aligned with both start and end to cache line size. Note: Rationale: This ensures that no other buffers/variables compete for the same cache lines.
EA_RTD_00087	When multicore feature is activated, MainFunction calls shall be scheduled for erase jobs with a period greater than the one described in the reference manual as a hardware limitation for the erase suspend-resume sequence. (e.g. on S32S2XX, this minimum time is defined in the reference manual as 5ms).
EA_RTD_00088	When both multicore and timeout features are activated, a large enough timeout value shall be used for jobs because of the HIPRIO semaphore shared usage that can cause job starvation on a core.
EA_RTD_00090	The Init job resets the driver's state(Example: clear SEMA4s, abort H↔W jobs, etc). The application shall ensure that, in multicore context, both instances of the driver are initialized before any other job is scheduled, driver is idle, or assume that the current running job may be aborted.
EA_RTD_00091	If the FLS synchronization feature is enabled, the FLS driver instances are not allowed to share any common flash sectors in their configuration.
EA_RTD_00104	In order to handle Flash ECC exceptions through ProtectionHook(), the user shall call the Flash_ReadEachBlock function inside a task that is started from the configurable callback guarded by FlsECCHandlingProtectionHook. Rationale: The task can be can be forcibly terminated by ProtectionHook() in case of an ECC exception.
EA_RTD_00105	If the Fls_DsiHandler() confirms that the ECC exception is caused by FLS driver read (function returns FLS_HANDLED_SKIP) then the protection hook shall return PRO_TERMINATETASKISR, in order for the Operating System to forcibly terminate the faulty task.
EA_RTD_00106	Standalone IP configuration and HL configuration of the same driver shall be done in the same project
EA_RTD_00107	The integrator shall use the IP interface only for hardware resources that were configured for standalone IP usage. Note:♦ The integrator shall not directly use the IP interface for hardware resources that were allocated to be used in HL context.
EA_RTD_00108	The integrator shall use the IP interface to a build a CDD, therefore the BSWMD will not contain reference to the IP interface
EA_RTD_00113	When RTD drivers are integrated with AutosarOS and User mode support is enabled, the integrator shall assure that the definition and declaration of all RTD functions needed to be called as trusted functions follow the naming convention Call<Function_Name>TRUSTE↔D(parameter1,parameter2,...) in Integration/User code. They need to visible in Os.h for the driver to call them. They will call RTD <Function_↔Name>() as trusted functions in OS specific manner.

9.1 Application's tasks

- It is responsibility of integrator/application to ensure that MCU-wide parameters like voltage supply etc. are according to and in limits specified in MCU documentation.
- Integrator/application is responsible to implement additional functionality that cancel any on-going erase/write Fls jobs if MCU conditions are not in such limits.

9.2 External memory

Flash driver shall support all Flash external memories that can be used via the QuadSPI Controller. These memories should:

- Fit to the LUT programming model
- Be electrically compatible with the QuadSPI IP
- The driver shall support initialization of such memories as well as configuring the command sequences for basic functionality (Flash Writes, Reads, Erase, etc)
- Rationale: In order to allow a wide array of Flash external memories that might be on customer boards, we allow the configuration of any memory that can be configured through the QuadSpi Controller
- To support various Flash external memories, the driver has callouts to allow configuring device specific features, for example the **FlsQspiInitCallout** field allows configuration of memory specific features during Init

Chapter 10

Tips for FLS integration

10.1 Synchronous vs. Asynchronous mode

- Asynchronous write mode works in the way, that `Fls_MainFunction()` just schedules the HW write operation and does not wait for its completion. In the next `Fls_MainFunction()` it is checked if the write operation is finished. If yes (depends on how often the `Fls_MainFunction()` is called), another write operation is scheduled. This process is repeated until all data is written. In this mode, `FlsMaxWriteFastMode/FlsMaxWriteNormalMode` values are ignored, maximum data can be written just by the maximum Tx buffer size of QSPI.
- When synchronous write mode is used, `Fls_MainFunction()` initializes write operation and also waits for its completion.
- The main differences between these two modes are in the time consumption and number of calls of the `Fls_MainFunction()`. The `Fls_MainFunction()` takes less time in asynchronous mode, but the whole write operation uses more `Fls_MainFunction()` executions.

- [ECC Management on Flash](#)

10.2 ECC Management on Flash

The section presents requirements that must be complied with when integrating the FLS driver into the application.

- [ECC Management on Qspi Flash](#)

10.2.1 ECC Management on Qspi Flash The ECC management on QSPI sectors is dependent on the external memory specific implementation.

The driver offers `FlsQspiEccCheckCallout`, called at the end of each read job to offer the possibility to the application to interrogate the external memory error status.

Depending on the hardware resources available on the external memory, the application can enable and implement the ECC callout in order to check any error bits available and return a failed value in order to mark the current job as failed.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

